

RESEARCH ARTICLE

Creating an API Ecosystem for Assistive Technologies Oriented to Cognitive Disabilities

RAQUEL HERVÁS¹, VIRGINIA FRANCISCO², EUGENIO CONCEPCIÓN¹,
ANTONIO F. G. SEVILLA¹, AND GONZALO MÉNDEZ¹

¹Facultad de Informática, Universidad Complutense de Madrid, 28040 Madrid, Spain

²Instituto de Tecnología del Conocimiento, Universidad Complutense de Madrid, 28233 Pozuelo de Alarcón, Spain

Corresponding author: Raquel Hervás (raquelhb@fdi.ucm.es)

This work was supported in part by the Digital Inclusion, Language and Communication (IDiLyCo) Project under Grant TIN2015-66655-R; in part by the Automated Composition of Personal Narratives as an aid for Occupational Therapy based on Reminiscence (CANTOR) Project under Grant PID2019-108927RB-I00 and Grant AEI/10.13039/501100011033; in part by the HumanAI-IU Project under Grant PID2023-148577OB-C22; and in part by the Dialogue Agents Relying on Knowledge-Neural hybrids for Interactive Training Environments (DARKNITE) Project, funded by the Spanish Ministry of Science, Innovation, and Universities, under Grant PID2023-146308OB-I00.

ABSTRACT This paper presents the development and implementation of an API (Application Programming Interface) ecosystem designed to support the creation of assistive technologies for individuals with cognitive disabilities. Leveraging the principles of microservices and an API-centered approach, this ecosystem enhances collaboration among developers, reduces the cost of creating accessible tools, and provides highly adaptable and customizable applications. The study details architectural decisions, including the use of GraphQL for flexibility, and demonstrates the benefits of a service-based architecture for digital inclusion. Case studies illustrate the practical applications, showing how this approach facilitates the development and promotes more accessible digital solutions. The findings suggest that this ecosystem can significantly reduce the time and resources needed to develop assistive technologies, potentially accelerating their adoption in domains such as education, healthcare, and daily living aids. The modular and flexible nature of the API ecosystem supports the rapid development of personalized assistive tools, fosters collaborative development, and ensures cross-platform compatibility. These features contribute to creating robust solutions that address diverse user needs across various devices. The ongoing evolution of the API ecosystem, including the integration of advanced management frameworks, promises further innovation and collaboration in assistive technologies. Future work will focus on implementing a comprehensive API management infrastructure to enhance scalability, security, and monitoring capabilities, including the integration and validation of the ecosystem with third-party applications, and so demonstrating its versatility and scalability in real-world scenarios.

INDEX TERMS Accessibility, API ecosystem, assistive technologies, cognitive disabilities, digital inclusion, GraphQL, microservices architecture, open APIs.

I. INTRODUCTION

Organizations are increasingly recognizing the critical role of digital assets as valuable resources that support the effectiveness of their business models. The ability to manage these assets distinguishes companies that integrate digital artifacts into their business value chain from those that continue to

rely on traditional approaches, relegating technology to mere support for business operations [1].

A well-grounded API strategy allows organizations to leverage data and services to speed up transformation and innovation processes, establish a basis for active digital services interchange, open up new, larger markets, and increase competitiveness. APIs can be viewed as digital ambassadors of an organization's capabilities.

In this context, the term API must be understood in a much broader context than the traditional programming

The associate editor coordinating the review of this manuscript and approving it for publication was Yizhang Jiang¹.

concept. Here, the API goes beyond the usual application interface exposed for a particular programming language, Web Services, and any other software interface. These APIs are designed and published in a protocol that can be seamlessly used by any application developed in any programming language. The exposition of open APIs enables organizations to provide uniform data and transaction interfaces not only to internal development teams, but also to external developers, partners, and customers, fostering innovation and collaboration.

Access to Information and Communication Technologies (ICT) plays an important role in improving the quality of life, independence, and development of . Much money and effort has been invested in adapting applications for people with disabilities and in creating new tools that facilitate digital inclusion; however, so far, this has not been done in the most efficient way possible. Typically, each developer creates applications from scratch, reinventing the wheel in several cases. One approach to reducing the cost of assistive technologies is to apply open-source principles to their development using and creating open APIs.

Applied to the research context, this approach can enable universities and research institutions to create new functionalities and value by developing new classes of applications based on new hybrid algorithms, models, and processes [2]. However, the adoption of such technologies and solutions in research remains minimal. Even within the same research field, researchers often implement their own solutions as monolithic systems, despite the potential for sharing reusable components that could benefit other researchers pursuing similar goals.

The use of a service-based architecture, along with an API-centered approach, is particularly suitable for promoting collaboration, which is highly desirable in the development of computer assistive applications. The primary goal of these systems is to provide tools that are adaptable to the personal needs of individuals with disabilities. This requires a flexible and extensible architecture, with extensibility being a key quality attribute. An extensible system is one in which the internal structure and data flow are minimally affected by new or modified functionalities. For example, recompiling or changing the original source code may be unnecessary when changing or updating a system's behavior, either by the creator or other programmers.

This approach allows users with disabilities to have tools that are highly adaptable to their personal needs, as the application can help them decide which services they need and how they should be configured. Additionally, designers and programmers of new computer assistive applications can integrate pre-existing services into their implementations, thereby saving significant costs in the research and development of these accessible technologies [3].

Within this context, the work presented in this paper describes an API ecosystem developed in the field of assistive

technologies for people with cognitive disabilities. This ecosystem has evolved from a set of monolithic applications into a flexible modular framework that supports a wide range of assistive tools.

The rest of the paper is organized as follows. Section II presents the background and related work in the field of assistive technologies and API ecosystems. Section III describes the architecture of the proposed API ecosystem, including its core components and design principles. Section IV provides detailed case studies that demonstrate the practical applications and benefits of the ecosystem. Section V discusses the interoperability and modularity of APIs and highlights their advantages and future potential. Finally, Section VI concludes the paper and outlines directions for future work.

II. BACKGROUND

In recent years, the concept of API economy has gained significant traction, changing its interaction with technology. APIs serve as bridges between various software applications, thereby enabling seamless communication and integration. This section provides an overview of the API economy, explores API management and evolution, highlights applications across various industries and academia, and discusses the specific roles of APIs in assistive technologies. In addition, we discuss various technical approaches to API design that support these developments.

A. API ECONOMY

Willmott and Balas defined an API Economy as “*the emerging economic effects enabled by companies, governments, non-profits, and individuals using APIs to provide direct programmable access to their systems and processes*” [4]. In contrast to the traditional approach of developing applications that address a specific need, every functionality in an API economy can be packaged as an API and made available to any interested consumer. This approach permits the development of a derived ecosystem, wherein APIs serve as fundamental elements for addressing a multitude of business needs that may not have been initially contemplated by the API designer.

The API Economy is structured around three main participants:

- **API Provider:** This is an entity—whether a person, team, or organization—that owns and makes certain assets available to consumers as services through an API under specific conditions.
- **API Consumer:** This is an entity—whether a person, team, or organization—that uses an API to build applications through which the can access the provider's assets.
- **End User:** This group of individuals utilizes the API based applications provided to them, taking advantage of the value-added services offered.

B. API MANAGEMENT AND EVOLUTION

API Management encompasses all aspects of governance, consumption, availability, and technical management related to APIs. This facilitates the control and management of numerous interactions among various applications and platforms. Furthermore, it enables businesses to securely transfer customer data across platforms.

Since their inception, APIs have undergone a significant evolution. In the early days of their development, APIs were primarily employed for software integration within the same organization. With the advent of Web APIs and cloud computing, APIs have become fundamental building blocks for modern applications, enabling interoperability, and innovation across industries.

The global API market size was valued at \$4.28 billion in 2023 and is projected to grow from \$5.42 billion in 2024 to \$34.17 billion by 2032 [5]. In today's digital landscape, APIs are not just prevalent, they are also dominant. Staggering 90% of developers utilize APIs, highlighting their integral role in modern technology. The API management sector is witnessing rapid growth, with a compound annual growth rate (CAGR) of 25.1%. Furthermore, APIs account for 83% of all internet traffic, underscoring their critical importance in the digital ecosystem [6].

Investment in APIs can be traced back to the early 2000s, with pioneers such as Salesforce and Google leading the charge. These early adopters paved the way for a new breed of companies, such as Stripe and Twilio, propelling the API economy while creating substantial shareholder value. Several examples of API economy development in the industry include Uber [7] and Google [8]. The industry has also played a leading role in academia. Most major IT companies have mature API ecosystems where developers from various companies can consume available APIs and create value-added services. Despite this, collaboration between industry and academia is not only possible but also enriching. This strategy was studied and discussed by Bonardi et al. [9].

C. API-BASED APPLICATIONS

APIs are ubiquitous in several industries. In finance, APIs enable secure transactions and access to banking services through platforms, such as Plaid and Stripe. In healthcare, APIs facilitate the sharing of electronic health records (EHR) and support telemedicine services. The entertainment industry uses APIs to stream content and provide real-time data for sports and media applications. These examples highlight the transformative power of APIs in driving innovation and improving the user experience.

API economies are pervasive in the industry. Nevertheless, initiatives to apply these methodologies in the context of scientific research and assistive technologies are lacking.

Kubler et al. [10] presented a framework that enables IoT (Internet of Things) service stakeholders (either service publishers or consumers) to join, contribute, and benefit from an open IoT ecosystem developed in the context of

an ongoing EU H2020 project called bioTope.¹ Another example is the FI-WARE EU FP7² project, whose main objective was to build open APIs for developers and providers, enabling the development and delivery of IoT applications through a common core platform architecture.

Another notable project is the W3C Web of Things (WoT)³ initiative, which aims to counter the fragmentation of IoT using web standards to enable interoperability across different IoT platforms and application domains. The WoT architecture includes a set of specifications for thing descriptions, binding templates, and scripting APIs, facilitating the creation and integration of IoT devices and services using a standardized approach.

Furthermore, the SmartAPI project, funded by the National Institutes of Health (NIH), focuses on creating a standardized framework for APIs used in biomedical research. This project emphasizes the importance of the interoperability and reusability of APIs across different research domains and aims to facilitate data sharing and collaboration among researchers [11].

D. ASSISTIVE TECHNOLOGIES

The World Health Organization reports that more than one billion people worldwide (16% of the global population) live with some form of disability [12]. This substantial segment of the population often encounters barriers when navigating the digital landscape. Inadequate accessibility limits access to information and services and impedes participation in online education, job opportunities, and social interactions. As our world becomes increasingly digital, ensuring digital inclusion is not only a matter of equity but also an economic imperative.

Assistive technologies (AT) encompass a wide range of devices, software, and equipment that help individuals with disabilities perform functions that might otherwise be difficult or impossible. In recent years, APIs have played a crucial role in advancing AT by enabling interoperability and customization.

Some significant contributions in this field include:

- **Speech Recognition and Synthesis APIs:** APIs, such as Amazon Polly and Google Text-to-Speech, allow developers to integrate voice control and interaction capabilities into applications, helping users with mobility or visual impairments.
- **Computer Vision APIs:** Optical Character Recognition (OCR) APIs, such as Google Cloud Vision API or Microsoft Azure Computer Vision, can extract text embedded within images, making them accessible to screen readers.

AT research has often focused on enhancing the usability and effectiveness of these technologies. For example, the GPII⁴ (Global Public Inclusive Infrastructure) initiative aims

¹<http://www.biotope-project.eu>

²<https://www.fiware.org/>

³<https://www.w3.org/WoT/>

⁴www.gpii.net

to develop a global infrastructure that integrates various assistive technologies through open APIs, promoting wider accessibility and customization. The development of GPII is currently in progress, and several projects are underway.

Although APIs offer powerful tools to enhance accessibility, they face their own set of challenges. To create a more inclusive digital environment, these APIs must be used for purposes and care. A significant challenge is ensuring that API developer portals themselves are accessible and readily available. Furthermore, the importance of visibility should not be overlooked. Clear naming conventions and effective search functions within API documentation are crucial, enabling developers to locate the features they need quickly and efficiently.

Cost can also be a barrier in implementing accessibility functions. Although some open API solutions exist, many accessibility features are provided through commercial APIs that require a subscription fee. This can be a limitation for smaller development teams or nonprofit organizations with tight budgets. Therefore, finding cost-effective solutions or promoting the use of comprehensive open-source accessibility APIs is essential. In addition, continuous maintenance and updates are vital for ensuring that these features remain functional and secure. Developers must consider the long-term costs of maintaining API integration to prevent future disruptions.

E. API ARCHITECTURES AND TECHNOLOGIES

From an architectural perspective, APIs are components of modular industrial architecture. They can be designated as either open (public) or private (internal) depending on the consumers to whom they are available. In contrast to open APIs, which are accessible to all, private APIs are only available to corporate applications or third-party consumers who have signed agreements with providers.

Several solutions can be used from a technological perspective. The most established methods are SOAP, REST, gRPC, and GraphQL. The technical approach presented in this paper combines GraphQL with Open API. Wittern et al. [13] presented an example of this approach.

The Open API strategy encourages a way of sharing data between entities in a trusted, timely, and open manner [14]. A clear example in the industry is the need for airlines and airports to share data [15]. This need is increasing annually. Conversely, other areas of interest, such as Artificial Intelligence based applications, cross-industry customer profiling and personalizing, and real-time operations, demand relevant, trusted, and timely data from different sources.

Open API connectivity enables innovation to thrive by opening up the traditional data silos of organizations to the rest of the world. This open ecosystem enables developers to build new solutions based on existing functionalities and exposed using these APIs, which would not be possible to build.

In line with this strategy of building APIs, the Linux Foundation oversees the redaction of the OpenAPI

Specification [16] in a collaborative approach. This specification is a formal standard for describing RESTful APIs and establishes how to enumerate resources and access parameters, using a document format that is readable by both humans and machines. This enables the use of tools for the automatic publishing, documentation, and consumption of OpenAPI-described APIs. This specification was employed to facilitate the publication of a unified and standardized access layer for our API ecosystem.

A different standard for describing and consuming APIs is GraphQL [17]. The GraphQL query language was initially developed by Facebook, and is now an open-source project overseen by the Linux Foundation. In technical terms, GraphQL is a query language, a mechanism for querying and manipulating resources described by an API. It is tightly coupled to a Schema Definition Language (SDL), which can be used to describe and publish the API.

GraphQL supports mutations and subscriptions (continuous updates to data), as well as queries, using the same language but with clearly separate semantics. The GraphQL query and schema definition languages are based on a type system in which the authors describe resources and their capabilities as a structured type hierarchy. This approach is more flexible and powerful than REST APIs; however, it is also more complex and presents a higher barrier to adoption.

This type of system enables authors to establish links between resources in a dynamic and recursive manner. Consumers of the API determine which resources and attributes are to be retrieved or modified, thus preventing the server from performing unnecessary work on resources that are not required by the client.

Since the query and schema definition languages are formalized, tools can process them automatically. Moreover, there are multiple implementations of GraphQL clients and servers that use specifications to automatically perform queries, mutations, and other services as requested by the client.

III. AN API ECOSYSTEM FOR ASSISTIVE APPLICATIONS ORIENTED TO COGNITIVE DISABILITIES

This section describes the architecture and services of the API ecosystem designed to support assistive applications for individuals with cognitive disabilities. The need for such an ecosystem arose from the desire to integrate and enhance pre-existing applications, ensuring interoperability, scalability, and the potential for the rapid development of new services.

A. MOTIVATION

The genesis of this project stemmed from the necessity to continuously evolve a collection of pre-existing, unrelated applications designed to address various accessibility problems for individuals with cognitive disabilities. Among the applications in question, two were identified as being of particular benefit to end users:

- NavegaFácil [18] is a web application tailored to users with reading comprehension difficulties. It enhances the

user's ability to navigate and understand the content of any web page by adding supplementary functionalities aimed at clarifying the meaning of the text. These enhancements include tools for finding synonyms, definitions, and images of unfamiliar words, thereby aiding the user in grasping the content more effectively. Rather than altering the original content of the web page, NavegaFácil enriches it with additional information that supports the comprehension of users with various reading disabilities or difficulties.

- AraTraductor [19] is a text-to-pictogram translator that relies on Natural Language Processing techniques to improve the obtained translations. This application processes plain text input through a syntactic analyzer, which informs the generation of translations that closely resemble those created manually by experts using ARASAAC pictograms. Implemented on Android, AraTraductor offers accessible text-to-pictogram translations via tablets and smartphones, providing alternative communication methods for users.

These tools turned out to be very useful for end users who needed alternative ways of communicating (pictograms) or some help in understanding complex texts. However, when we attempted to develop these tools further or even combine them (for example, adding text-to-pictogram functionality to NavegaFácil), we concluded that interoperability was not technically feasible in their original state. Consequently, it became clear that adapting their architectures was a necessary step.

This approach was adopted to ensure that the system would be interoperable and scalable. To achieve this objective, the original monolithic applications were restructured as a set of resources accessible via application programming interfaces (APIs). We adhered to the leading principles of resource-oriented architectures [20] to ensure a robust solution design. The newly implemented API ecosystem not only facilitates the integration of existing functionalities but also supports the development of new, higher-level services that can leverage the basic services provided by these APIs.

Technologies such as GraphQL in conjunction with the OpenAPI Specification were employed to guarantee that the system is flexible, scalable, and easily maintainable. The objective of this new architectural framework is to create a more unified and adaptable ecosystem that can more effectively address the evolving needs of users with cognitive disabilities. It is anticipated that this will facilitate rapid and efficient development of innovative assistive applications.

The newly implemented architecture, the API services, and its various access layers are described in detail in the following subsections.

B. ARCHITECTURAL APPROACH

The architecture of existing assistive applications has been designed to make the collaboration between them a

challenging task. This is because almost every system duplicates several common functions.

If each system is decomposed into more granular components, such as microservices [21], [22], these components can be utilized independently. Additionally, each microservice would possess sufficient autonomy to undergo independent evolution in response to new requirements, without compromising the integrity of the overall architecture. The most notable outcome of this approach is the potential to construct hybrid coarse-grained services by combining existing microservices. The proposed system uses the most effective components to construct a collaborative generation architecture.

Our methodology entails the development of a multitude of functionally specialized and modular services. Each service can be integrated into a larger and fully operational solution tailored to the specific needs of a given user base and scenario.

Fig. 1 illustrates the evolution of the architecture, which is explained in the following subsections.

1) WEB SERVICES AND PROTOCOLS

The initial step in the development of the new architecture was based on the separation of work packages or units. Each project and research product is autonomous and operated according to the specific requirements of the project and its context. However, to facilitate future reuse, it is necessary to employ a standardized and sufficiently open communication protocol. In our view, this implies the use of web services operating on top of the Hypertext Transfer Protocol (HTTP).

Web services have several advantages. They provide a common platform that is technology-agnostic for clients, eases the isolation of applications, and simplifies the maintenance phase by separating concerns. Despite some legacy projects returning plain text or HTML fragments, recent and future projects have adopted XML and JSON standards, ensuring consistency in data encoding and URL structuring.

2) GATEWAY SERVER

For these services, a gateway server (NGINX) serves as the entry point for the entire API subsystem. The gateway server performs several functions, including security, encapsulation, logging, and load balancing. Furthermore, it employs a REST-inspired URL scheme that organizes services according to the objects on which they operate, rather than by when or by whom they were developed. The URL scheme is routed to the appropriate services via request rewrite rules in the NGINX. In addition, it establishes a common transfer and encoding standard, once again inspired by REST.

Communication is conducted exclusively in JavaScript Object Notation (JSON) format, irrespective of whether the outcome is a successful transaction or error. Information is provided in the form of JSON objects, accompanied by relevant HTTP status codes. Moreover, the input to web services is also in the form of JSON objects, with simple

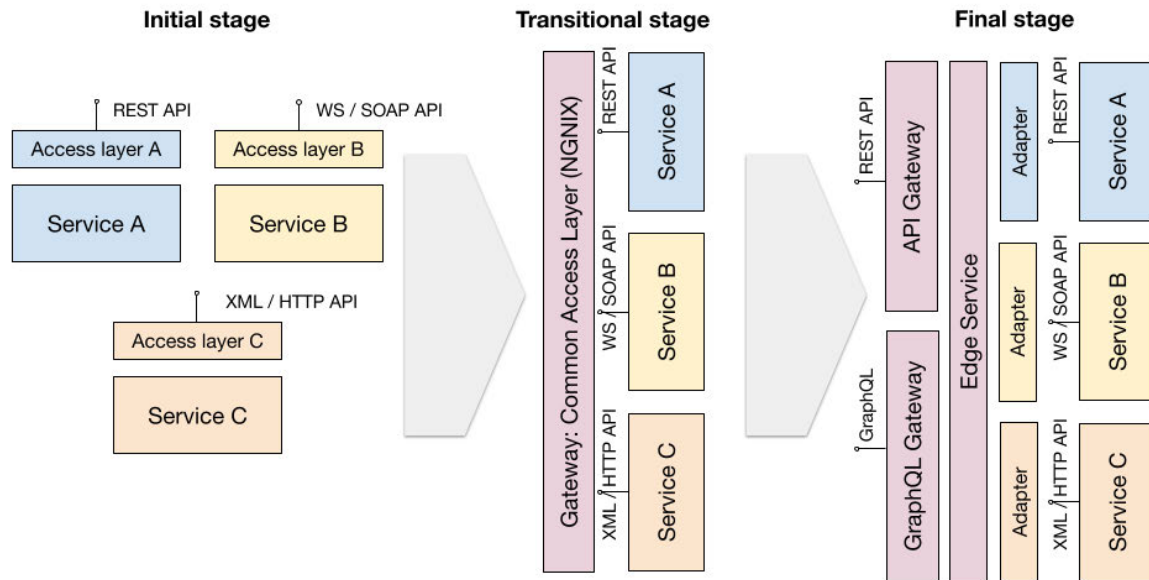


FIGURE 1. Architectural evolution: from disaggregated services to an API ecosystem.

parameters related to the service operation encoded in the URL if feasible.

To address the issue of non-standard older services, LUA scripts embedded in NGINX were employed for the conversion of requests and responses. This ensured seamless integration and uniformity. The complete specifications of the appropriate URLs, body formats, responses, and errors are accessible in a publicly available OpenAPI document, thereby facilitating the discovery and enhancing usability.

3) LAYER OF SERVICES

A further extension of this architectural model that is currently being implemented is the addition of a third tier. This additional layer of services, which is integrated into the gateway but is not exposed to the public, implements common natural language algorithms and data that may be required by numerous internal services. Consequently, the development of a new component that addresses a specific issue for a particular target audience can be expedited, as common processing tasks may have already been resolved by an internal service. As the service was developed using a specific issue in mind, it is possible to use the most appropriate libraries, algorithms, and data to resolve it. For example, several services may require access to the dependency parse of a sentence. There is a plethora of potential solutions to this problem, which vary in terms of their quality and suitability for a specific problem or language. However, our internal Natural Language Processing (NLP) service is used to address this issue. The service is capable of utilizing a diverse range of parsing libraries, incorporating the most appropriate data, and combining solutions into the optimal parse. Thus, the aforementioned services can be utilized to obtain a high-quality dependency parse in a matter of minutes. Furthermore, this approach enables the uniform

processing of language across the entire ecosystem, thereby enhancing the predictability of the solutions we provide for common problems.

An additional GraphQL server that is now accessible to the public was introduced in conjunction with the OpenAPI server. GraphQL integration allows for a combination of querying, data structure, and internal NLP services, thus eliminating the necessity for multiple round trips and reducing latency.

While the API cannot be considered fully REST-compatible, the design guidelines and best practices for REST systems were followed. In essence, REST prompts us to conceptualize our services in terms of resources (noun-like entities to be acted upon), their attributes, and the HTTP verbs that can be performed on them.

As most existing services do not adhere to this system, it is necessary to implement certain adaptations to ensure compatibility. LUA scripts are utilized for rewriting incoming Uniform Resource Locators (URLs) and converting parameters and data as required. In certain cases, an intermediary in-memory midpoint is utilized to facilitate communication with upstream services. However, the LUA scripts are executed by the gateway itself, utilizing NGINX LUA extensions, which results in minimal added latency.

The GraphQL server, which employs Ariadne, uses a distinct approach to parameter and data-carrying, diverging from that of the OpenAPI gateway. Direct communication with upstream services is employed to reduce latency, although this approach may result in some redundancy.

C. API SERVICES

The objective of our API architecture is to facilitate communication and information access in the digital world for groups of individuals who, due to their functional diversity,

may encounter difficulties when attempting to communicate with others or to access digital information through natural language. In light of our previous experiences in related fields, our objective is to develop technological solutions that are both simple and highly configurable to facilitate digital inclusion for these individuals. The architecture is organized into three key areas of accessibility and natural language processing: Text Simplification—which automatically or semi-automatically rewrites complex texts more simply; Augmentative and Alternative Communication—which uses alternative systems to facilitate communication for users with functional diversity; and Sentiment Analysis—that identifies whether a text is emotionally ambiguous.

These fields are currently the subject of active research with promising results. However, their combined application in enhancing digital inclusion has not yet been thoroughly explored. For example, textual content into an accessible format requires summarizing and rewriting text in a simple and clear language that individuals with diverse disabilities can comprehend. This process involves coordinated use of multiple technologies: text simplification, which rephrases complex text in a more accessible manner; sentiment analysis, which detects emotional ambiguities; and Augmentative and Alternative Communication (AAC), which expresses content through alternative communication methods such as pictograms.

Fig. 2 provides a comprehensive diagram of the API architecture, illustrating the series of APIs developed in the aforementioned fields, some of which leverage legacy systems. These APIs can be combined to create new services and add value. Table 1 lists all the microservices, describing each and assigning a code for future reference.

The following subsections present a detailed account of the architectural divisions and the associated APIs within each field.

1) TEXT SIMPLIFICATION

The objective of text simplification is to adapt texts for readers who experience difficulty in understanding them. This may include non-native speakers, the elderly, and individuals with dyslexia or other cognitive disabilities. To facilitate automation or semi-automation of text simplification, rule-based systems, and systems based on aligned statistical corpus analysis (original-simplified) have been developed. A variety of techniques have been employed within these systems, including lexical substitution of uncommon words with common words, transformation of passive sentences into active ones, reference resolution, changes at the discourse level, syntactic simplification, and others. Although text summarization is not considered a sub-field of text simplification in NLP, it is included here because the generation of summaries is a fundamental process for determining which aspects of a document or set of documents should be simplified. New wording should reflect these aspects and eliminate all information that is not relevant to the user.

TABLE 1. List of microservices, showing the API that contains the microservice, a code for identification in the text, and a description of their purpose. These are further categorized into various application areas.

Text simplification		
Simplification API	SIM1	Verifies whether a word is simple (part of an easy-to-understand subset of the vocabulary) or not
	SIM2	Gives a list of synonyms for the word, which can be used to provide alternatives for complicated terms.
	SIM3	Gives a list of antonyms, which can also help understand a term by contrast, providing negative examples of its meaning.
	SIM4	Provides definitions for a word.
	SIM5	Provides English translations of the word.
	SIM6	Converts (translates) a word into a simple alternative or synonym for it.
	SIM7	Finds easy hyperonyms for a word.
	SIM8	Finds easy hyponyms for a word.
	SIM9	Returns examples of the use of a word in context.
Summarization API	SIM10	Creates a short extractive summary from a given text.
Rhetorical Figures API	SIM11	Creates metaphors for a word, to aid in understanding its meaning.
	SIM12	Finds similes for a word, to aid in understanding its meaning.
Augmentative and Alternative Communication		
Pictogram API	AAC1	Gives a pictogram for the meaning of a word.
	AAC2	Gives a series of pictograms for the meaning of a text.
	AAC3	Finds the meanings that can be associated to a pictogram.
Emotional analysis		
Emotions API	EMO1	Gives the emotion most strongly associated with the word, along with the degree to which it is associated.
	EMO2	Gives the consensus emotion, an emotion chosen by consensus by all evaluators for the given word.
	EMO3	Grades all basic emotions with a number which represents how strongly the word evokes that emotion.
	EMO4	Gives a list of emotion-evoking words in a supplied text.
Auxiliary Natural Language Processing API		
NLP API	NLP1	Returns the results of morphological analyzes of the word.
	NLP2	Returns different grammatical analyzes of the text (separation into sentences, named entities, noun phrases, ...).
	NLP3	Returns various information about the Wordnet synset, such as a list of lemmas that belong to the synset, and other synsets that are hypernyms or hyponyms of the requested one.

The developed architecture included three APIs in this field. The Simplification API contains several services that implement basic functionalities for text simplification, such as synonym or definition searches and translation to a simpler vocabulary. Many of the services of this API are reimplementations of parts of the NavegaFácil legacy system.

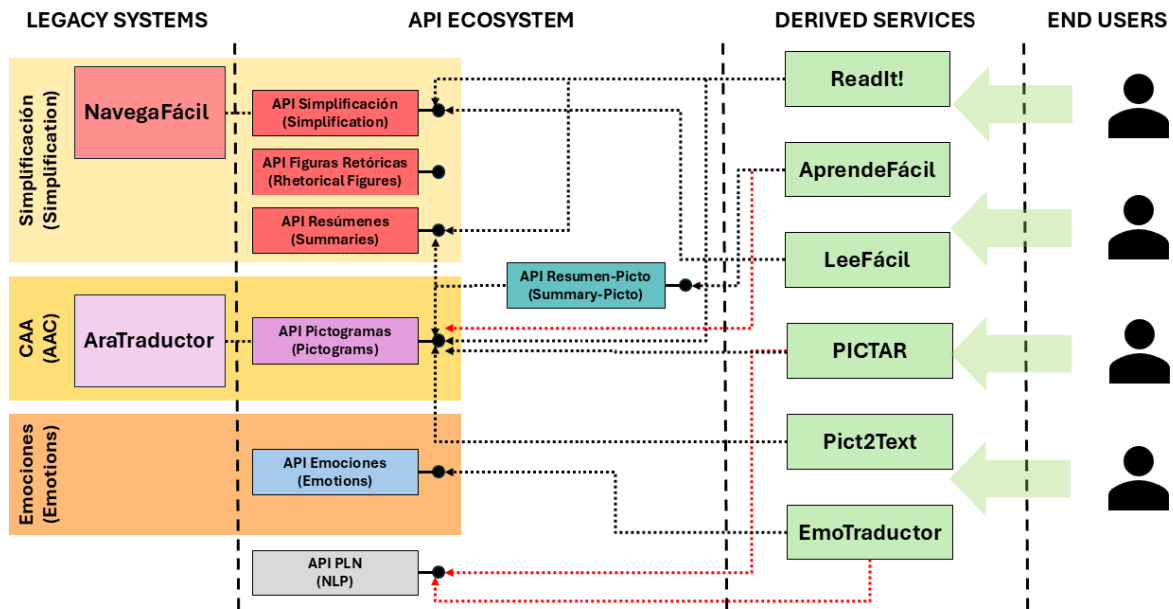


FIGURE 2. Complete diagram of the developed API ecosystem.

The Rhetorical Figures API includes services that look for relationships between concepts, which can be used to find known concepts to the user when trying to explain unknown information. Finally, the Summarization API contains a service to create summaries from the given texts. A complete list of services offered by these APIs can be found in Table 1.

2) AUGMENTATIVE AND ALTERNATIVE COMMUNICATION

Augmentative and Alternative Communication (AAC) systems are designed for users who are unable to utilize natural language as a result of either temporary or permanent impairments, or for whom the use of natural language presents a considerable challenge. In all cases, the objective of AAC is to provide an alternative to natural language, thereby enabling users with communication impairments to effectively communicate in their daily lives. Pictograms are one of the most frequently utilized resources within the AAC domain. A pictogram is a visual symbol representing an idea or concept that facilitates communication across language barriers. For individuals experiencing difficulties with language, pictograms serve a dual function: they facilitate the expression of ideas and feelings and provide a of interpreting, understanding, and converting thoughts into visual representations. The range of concepts that pictograms can depict is extensive, encompassing objects, animals, people, emotions, actions, and grammatical elements.

In our architecture, we develop a dedicated API for this field. The Pictogram API includes several services that facilitate the use of pictograms such as text-to-pictogram translation and pictogram-to-text translation. A comprehensive list of the services provided by the API is presented in Table 1.

3) SENTIMENT ANALYSIS

The complexity of human emotions provokes the inability of many people with disabilities to correctly understand them. Individuals with Autism Spectrum Disorders (ASD) or Asperger's Syndrome (AS) face significant challenges in perceiving emotions, which are crucial for their emotional and social development. The automatic detection of emotional content in text can assist individuals with disabilities to better understand the text, thereby avoiding unnecessary emotional ambiguities.

The architectural design outlined here encompasses a set of APIs that facilitates the functionality associated with this particular field. The Emotions API includes several services that implement sentiment analysis functionalities such as finding the emotional words in a text and deciding the emotion that a text or word is conveying. The complete list of services offered by this API is presented in Table 1.

4) AUXILIARY NATURAL LANGUAGE PROCESSING API

Furthermore, auxiliary language analysis functionalities have been developed to support a variety of services and applications. These have been designed for use across multiple development projects and are therefore considered to be transversal. Specifically, the following services have been developed on spaCy⁵ library and WordNet lexical resource [23]:

- Word morphological analysis: Analyzes a word and returns its morphological analysis.
- Grammatical analysis of text: Provides grammatical analysis of a given text, including sentence separation,

⁵<https://spacy.io/>

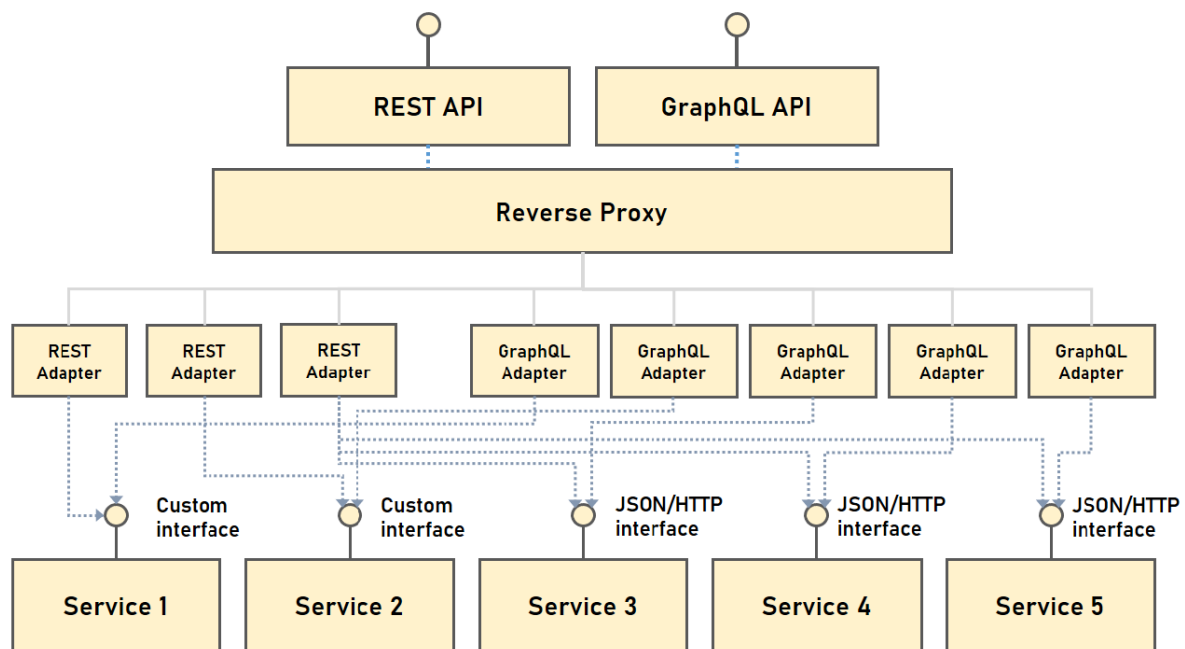


FIGURE 3. Access layers.

named entity recognition, noun phrase identification, etc.

- **Wordnet Synsets:** Returns various information about Wordnet synsets, such as a list of lemmas that belong to the synset, and other synsets that are hypernyms or hyponyms of the requested one.

D. ACCESS LAYERS

As shown in Fig. 3, two layers of access to the services offered by the aforementioned APIs were implemented. The REST API adheres to the OpenAPI Specification, and GraphQL is also supported. This dual-layer approach ensures both flexibility and efficiency, thereby addressing diverse user needs and technical preferences that may arise. The Open API layer is appropriate for uncomplicated, RESTful access to resources, whereas the GraphQL layer offers a more versatile querying mechanism that can integrate multiple data sources in a single request. This duality ensures optimal accessibility and usability for developers with disparate requirements.

1) REST API (OPEN API)

When developing the final version of the API, we considered various approaches for defining the underlying URL structure. One potential approach would be to adopt a flat layout with all endpoints situated at the top level. An alternative approach would be to categorize the endpoints according to their thematic content, reflecting on the projects under which they were developed. However, from the perspective of end users, this may be of little consequence, given that the

objective is to enable users to combine and utilize different services to leverage our infrastructure for purposes that fall outside the scope of our predefined packages. From the perspective of the user, it was more practical to structure URLs based on the input objects and abstract resources on which the services operate.

This concept has led to the development of an API that is divided horizontally, with services that act on words as separate tokens or entities distinguished from those that operate on full texts, which are considered distinct entities:

- **Word Services:** The first level in the URL is “word” (in Spanish “palabra”) or text (in Spanish “texto”). The next level in the URL is the actual word, and therefore the URL `/palabra/<word>` becomes the representation of the abstract resource corresponding to that particular word. The next bit in the URL thus represents the attribute to be queried, such as the primary emotion associated that word or whether it is simple. Since our APIs are all read-only, the HTTP verb to be used is always GET for words, and requests URIs become very simple.
- **Text Services:** Texts are not stored in our services because of their infinite variety. Clients must send the complete text to the service, which does not align well with GET requests. Thus, texts are posted to a single endpoint `()` in a JSON object. Subsequent segments in the URL specify the properties to be queried, such as the primary emotion or summary.

Once the inputs to the system have been defined, only the outputs need to be defined. achieve this, three distinct

```

/nil-ws-api
├── /palabra/<word>.....The word is a part of the url
│   ├── /es_sencilla.....SIM1
│   ├── /sinonimos.....SIM2
│   ├── /antonimos.....SIM3
│   ├── /definiciones.....SIM4
│   ├── /traducciones.....SIM5
│   ├── /conversion_a_facil.....SIM6
│   ├── /hiperonimos.....SIM7
│   ├── /hiponimos.....SIM8
│   ├── /ej_uso.....SIM9
│   ├── /metaforas.....SIM11
│   ├── /similes.....SIM12
│   ├── /pictograma.....AAC1
│   ├── /significado.....AAC3
│   ├── /emocion_mayoritaria.....EMO1
│   ├── /emocion_consensuada.....EMO2
│   ├── /grados_emociones.....EMO3
│   ├── /analisisMorfológico.....NLP1
│   └── /synsets.....NLP3
├── /texto.....Receives text by post
│   ├── /resumen.....SIM10
│   ├── /pictogramas.....AAC2
│   ├── /palabras_emocionales.....EMO4
│   └── /analisisGramatical.....NLP2

```

FIGURE 4. OpenAPI access layer URL scheme.

categories of response were identified. A successful operation returns a 200 OK status code, accompanied by a JSON object containing the queried properties. In the case of a property not being found, a 404 error code is returned, accompanied by a standard “not found” text. If the requests are malformed, a 400 status code is issued without any accompanying message. Finally, if the server encounters an unanticipated error, a 500 status code is generated.

The OpenAPI specification for our API is public and available at <https://holstein.fdi.ucm.es/nil-ws-api/>. Fig. 4 shows the mapping of entry points in the OpenAPI layer to the underlying microservices.

Listing 1 provides examples of API usage, including GET and POST requests with their corresponding JSON responses. The first example is a GET petition, with the url, and the JSON object it returns. This petition asks about the resource “profesor” (teacher in Spanish), and the attribute to query is “grados_emociones”, the strength of each basic emotion as elicited by this word. We can see that the response object is a dictionary with each emotion as a key and its degree as values. In the second example, a text is queried. The resource (the text) does not appear in the URL but rather is sent in the body of the POST request. The attribute to query, which is the last segment in the URL, is “palabras_emocionales”, and the response is a list of emotion-carrying words in the text (the key “palabras” in the response object) along with the average degree for emotions in the text (the key “emociones”).

```

GET /nil-ws-api/v1/palabra/profesor/grados_emociones
→ nes
{
  "Alegria": "2.2",
  "Asco": "1.3",
  "Enfado": "1.57",
  "Miedo": "1.43",
  "Tristeza": "1.3"
}

POST /nil-ws-api/v1/texto/palabras_emocionales
{
  "texto": "Las abejas me persiguieron hasta mi
→ casa, pero no me pudieron picar porque
→ corri mucho"
}
→
{
  "emociones": {
    "Alegria": "2.57",
    "Asco": "1.69",
    "Enfado": "1.61",
    "Miedo": "2.35",
    "Tristeza": "1.25"
  },
  "palabras": [
    "abejas",
    "persiguieron",
    "casa",
    "picar",
    "corri"
  ]
}

```

LISTING 1. Examples of use of the OpenAPI access layer (translated into English in Listing 4).

2) GRAPHQL

After establishing the Open API architecture, we adopted GraphQL, a flexible standard for querying structured information over HTTP. GraphQL allows users to transparently combine services and operations within our network, minimizing the need for multiple queries. Our GraphQL architecture mirrors the resource-centric model of REST but enhances it by defining DataTypes for responses. This removes redundancy and ensures consistent service application.

When defining our GraphQL architecture, we realized that most of the work already done could be ported to the new model. Like REST, GraphQL is resource-centric; therefore, our system again consists of Word and Text objects. However, instead of simply defining response codes and a JSON model, we also added DataTypes for the responses in our GraphQL specification. For example, emotions were no longer plain, structure-less JSON data returned by querying the emotion attributes in words or texts, but actual datatypes were part of the word or text objects.

The benefits of this approach were evident from the outset and included elimination of unnecessary processes and services. A significant number of word-based services also provided text versions that acted on all words in the text. This resulted in a considerable number of services having duplicated code for text tokenization, with inconsistencies

```

POST /nil-ws-api/graphql
{
  "query": "{
    palabra(s: \"abeja\") {
      pictograma {
        id
      }
      gradosEmociones {
        Miedo
      }
    }
  }"
}
→
{
  "data": {
    "palabra": {
      "pictograma": {
        "id": "2239"
      },
      "gradosEmociones": {
        "Miedo": 3.53
      }
    }
  }
}

```

LISTING 2. Example of use of the GraphQL access layer (translated into English in Listing 5). Note how the shape of the request body and response match. The predictability of the response data is a strong point of GraphQL.

in the approach taken. With GraphQL, the Text object has a property that is the list of its words. Since words are also GraphQL datatypes, their properties can again be queried, so automatically all services available for words are available for texts as well. Since now, tokenization is done at the GraphQL server, it is more consistent and can use the best NLP tools available.

The GraphQL Schema for this access layer is shown in Fig. 5, with usage examples provided in Listing 2. Access to our GraphQL API is available at <https://holstein.fdi.ucm.es/nil-ws-api/graphql/>, and if visited with a browser, an interactive playground⁶ is served which can be used to explore the API.

3) PRACTICAL USE CASES

While Open API provides simplicity and ease of use, GraphQL offers flexibility and efficiency for complex queries. Depending on the use case, developers can choose the most suitable access layer to balance the performance and functionality. The Open API access layer is ideal for applications requiring predefined, straightforward access to resources using a RESTful approach. For example, retrieve the primary emotion associated with a word or a summary of a text. In contrast, the GraphQL access layer is suitable for applications that require complex queries that involve combining multiple services. For instance, simplification, sentiment analysis, and pictogram translation of a given text in one request (see Listing 3).

Input:

```

{
  "query": "{
    texto(texto: \"Me siento apenado cuando llueve.\") {
      simplificado
      gradosEmociones {
        tristeza
      }
      pictogramas {
        url
      }
    }
  }"
}
Response:
{
  "data": {
    "texto": {
      "simplificado": "Me siento triste cuando llueve.",
      "gradosEmociones": {
        "tristeza": 0.87
      },
      "pictogramas": [
        {"url": "pictogram1.png"},
        {"url": "pictogram2.png"}
      ]
    }
  }
}

```

LISTING 3. Obtaining simplification, sentiment analysis, and pictogram translation of a given text in one request using GraphQL (translated into English in Listing 6).

The flexibility of the Open API and GraphQL layers ensures that developers can choose the most suitable approach for their specific needs, enabling the rapid and efficient development of innovative assistive technologies.

E. INTERACTIONS BETWEEN APIS

IV. THE ECOSYSTEM IN PRACTICE: CASE STUDIES

The API architecture outlined previously has been instrumental in bringing advanced techniques closer to with cognitive disabilities. This is achieved through a series of highly configurable applications that leverage these services to meet the specific needs of individual users. All these applications were developed following MeDeC@, a Methodology for the Development of Computer Assistive Technologies for individuals with Autism Spectrum Disorders (ASD) [3]. The aim of this methodology is not to design for a broad group of users but to design highly customizable tools so that they can be easily adapted to specific situations and small user groups, which is very important in the field of accessibility. For example, whereas many applications are configured with general profiles that try to cover all the possibilities for a specific group of users, capabilities and individual preferences are not the same for two different people even if they share the same disability.

A fundamental design principle of MeDeC@ is the use of a Service-Oriented Architecture (SOA) for developing computer assistive applications. An essential part of this implementation is to determine the services that will

⁶<https://github.com/prisma-labs/graphql-playground>

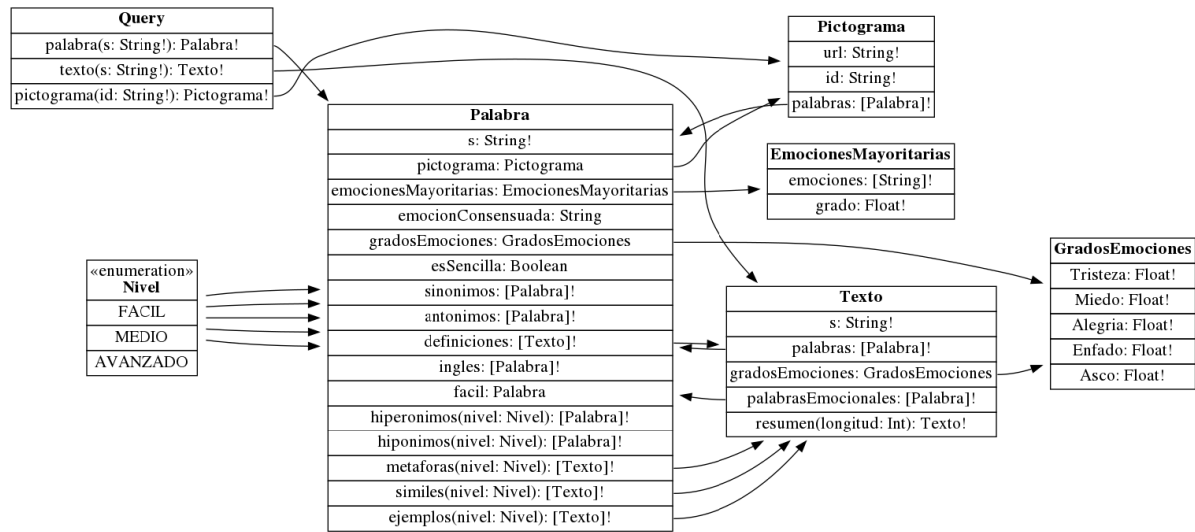


FIGURE 5. Type Schema for the GraphQL access layer (translated into English in Figure 6).

encapsulate the functionalities comprising the application. For instance, a text simplification application might include services for the lexical simplification of individual words, syntactical simplification of sentences, or text summarization. These components can then be assembled into distinct functionalities in the final application. This architecture facilitates easy customization for each user by allowing the selection and activation of relevant services as needed.

In the rest of this section, we present six applications implemented using the API ecosystem. Some of them were the original sources of some of the microservices in the ecosystem, whereas others have benefited from these pre-implemented services. The modularity and flexibility of the microservices architecture allowed these applications to easily configure new services as needed and update the latest versions of each functionality without altering their source code or version.

A. PICTAR: A TOOL FOR EDITING PICTOGRAM CONTENT BASED ON TEXT-TO-PICTOGRAM TRANSLATION

PICTAR [24] is an Augmentative and Alternative Communication System (AACs) designed for people with Autism Spectrum Disorder (ASD). working with pictograms and developing materials for special education. The application not only allows the creation of materials based on pictograms in a user-friendly manner but also includes text-to-pictogram translation capabilities. Evaluations with special education experts and users with cognitive disabilities who communicate through pictograms have demonstrated its usefulness for both teachers and students. PICTAR enhances literacy, communication, and motivation skills for students with ASD. PICTAR can be freely accessed at <http://hypatia.fdi.ucm.es/pictar/>.

PICTAR utilizes services from both the Pictogram API and the NLP API, as shown in Table 2. By leveraging

these modular services, PICTAR can easily integrate updates and new features without significant redevelopment, thereby demonstrating the flexibility of the microservice architecture.

B. LeeFácil: MOBILE ASSISTANT FOR THE EASIER INTERPRETATION OF TEXTS

LeeFácil [25] is a mobile application designed for individuals with cognitive disabilities who have difficulty reading natural language texts. It provides support for text comprehension on the go, without a computer. Users can take a picture of the text they do not understand and activate various within the application, such as transforming the text into pictograms, simplifying complex words, and obtaining word definitions. LeeFácil can be downloaded from Google Play Store.

Each functionality of LeeFácil leverages services from the Simplification API and Pictogram API, providing great flexibility for introducing new features. The detailed services used in the application are listed in Table 2. This modular approach allows LeeFácil to adapt its features to the specific needs of its users, thereby demonstrating the adaptability and scalability inherent in the microservice architecture.

C. AprendeFácil: IMPROVING READING COMPREHENSION THROUGH RHETORICAL FIGURES

AprendeFácil [26] is a web application designed to . The solution in would be to look up the unknown concept in a dictionary. However, this is not always a solution for users with cognitive disabilities, dictionary definitions complex vocabulary and ideas that are difficult to understand. In AprendeFácil, users can any difficult word, and get a definition simpler vocabulary, pictograms and comparisons to other simple concepts using similes and metaphors. AprendeFácil can be accessed freely at <https://holstein.fdi.ucm.es/tfg-analogias/>.

TABLE 2. List of the services used in each of the use cases presented.

API	Service	ReadIt	AprendeFácil	LeeFácil	PICTAR	Pict2Text	EmoTraductor
Simplification API	SIM1	X					
	SIM2	X		X			
	SIM3	X		X			
	SIM4	X	X	X			
	SIM5	X					
	SIM6	X	X				
	SIM7	X		X			
	SIM8	X		X			
	SIM9	X		X			
Summarization API	SIM10	X					
Rhetorical Figures API	SIM11		X				
	SIM12		X				
Pictogram API	AAC1		X	X	X		
	AAC2	X		X	X		
	AAC3					X	
Emotions API	EMO1						X
	EMO2						X
	EMO3						X
	EMO4						X
NLP API	NLP1		X	X	X		X
	NLP2			X	X	X	
	NLP3		X	X	X		

AprendeFácil uses services from the Simplification API to find words for its definitions, the Rhetorical Figures API to definitions, and the Pictograms API to include pictograms. The specific services that were used are in Table 2. AprendeFácil seamlessly integrate new simplification and rhetorical figure services as they become available, enhancing its capability to aid comprehension.

D. ReadIt: A BROWSER PLUG-IN TO ENHANCE WEB NAVIGATION

ReadIt [27] is a web browser extension that enhances web navigation by functionalities to facilitate the understanding and simplification of written information. ReadIt can be downloaded from the Google Chrome Web Store.

ReadIt uses the same API services as LeeFácil, along with additional services, demonstrating the flexibility of the technological solution presented. A detailed list of the services used is presented in Table 2. The use of microservices allows ReadIt to offer a highly customizable experience, enabling the addition or removal of features based on user feedback and emerging needs.

E. Pict2Text: PICTOGRAM-TO-TEXT TRANSLATOR

Pict2Text [28] is a web application for translating texts written with pictograms into natural language. It helps individuals who communicate using pictograms to translate their messages into text, facilitating communication in various settings (e.g. a shop, an excursion, or a restaurant). Pict2Text can be accessed at <https://holstein.fdi.ucm.es/tfg-pict2text/>.

Pict2Text was by leveraging the Pictogram API services initially created for PICTAR, as shown in Table 2. This reuse of established services illustrates the efficiency and resource

optimization provided by the microservice architecture, enabling the rapid development and deployment of new tools.

F. EmoTraductor: A WEB APPLICATION FOR EMOTIONAL TEXT ANALYSIS

EmoTraductor [29] is a web application that automatically detects the presence of the five basic emotions (joy, sadness, disgust, fear, and anger) in a given text. Not only does the application show the main emotions transmitted in a text using emoticons and colors, but it also highlights the words in the text that are considered emotional and therefore contribute to the detected emotions. This tool is specifically intended for users with Asperger's Syndrome, who may have difficulty recognizing and expressing their emotions. In these cases, it is very useful for them to have an "emotional translator" capable of suggesting the emotions transmitted by the text they are reading or writing. EmoTraductor can be accessed at <http://sesat.fdi.ucm.es/traductor/>.

EmoTraductor exclusively uses the services of the Emotional API detailed in Table 2. While these services are not currently used in any other application, they can be integrated into tools such as ReadIt to provide an emotional analysis of web content, enhancing users' emotional understanding. Additionally, applications such as AprendeFácil can benefit from these services by offering emotional definitions of complex concepts. applications could include emotional assistants for creative writing or sentiment analysis tools for social media, demonstrating the versatility and potential of Emotional API services to diverse needs.

G. DISCUSSION OF API INTEROPERABILITY

The case studies presented above underscore the substantial advantages of the API ecosystem and microservice architecture in developing assistive technologies for users with

cognitive disabilities. The modularity and flexibility of this approach facilitates the creation of highly customizable and adaptable applications tailored to meet the unique needs of individual users.

API interoperability is a critical factor in this ecosystem, enabling seamless integration and interaction between different services. By leveraging reusable services, developers can efficiently create and update applications, ensuring that the latest functionalities are readily available without extensive rework. This interoperability allows the combination of various microservices, such as text simplification, pictogram translation, and emotional analysis, to build comprehensive solutions that address diverse user requirements.

Furthermore, the ability to incorporate new services into existing applications without significant code changes demonstrated the robustness and scalability of the microservice architecture. This not only enhances the development process, but also provides with tools that significantly improve their daily lives. Modular design ensures that applications can evolve alongside technological advancements and user feedback, fostering greater independence and empowerment for individuals with cognitive disabilities.

V. DISCUSSION

One of the main advantages of developing an API ecosystem is its scalability and innovation. This approach allows partners, and potentially any developer if the APIs are open-source software, to create new applications by leveraging the data and functionalities provided. Essentially, when an organization offers an API to its partners, it provides a collection of dedicated URLs that return data-based resources that can be directly processed by other applications and APIs. This capability enables organizations to scale quickly by accessing third-party data and services through APIs.

The decisions made for our API ecosystem were influenced not only by theoretical architectural principles but also by pragmatic considerations. A clear example is the decision not to adopt a pure REST implementation. In many cases, adapting legacy systems to canonical RESTful interfaces introduces limitations and transformations that would impair the capabilities of the solution. Instead, using a GraphQL interface provides a more flexible approach, particularly for consumers. The main advantage of GraphQL is its ability to adapt the response according to the specific information required by the consumer. It also provides a navigation-like query model that operates as a resource repository, which can be integrated as a back-end commodity for higher-level services.

Furthermore, the API ecosystem has proven to be highly flexible in terms of the configuration and personalization of assistive tools. The availability of services that solve small accessibility problems has simplified the design process. Once the main operational blocks are available, the remaining choices are mostly about interface and interaction design. This approach facilitates the rapid development of new tools that address similar issues but need to be personalized

for different user groups. As Harper [30] pointed out, the separation between the user interface and functionality allows universal access to applications, as the interface can be adapted to the user without modifying the underlying functional code. Therefore, we believe that exploring this type of architecture can significantly enhance the development of assistive technologies and reduce costs, effort, and development times.

In addition, the API ecosystem fosters collaborative development, allowing multiple developers and organizations to work together to build more integrated and robust solutions. Providing a standardized set of services enables diverse teams to contribute to and enhance the overall ecosystem, leading to a richer set of functionalities and innovations.

The versatility of these APIs was further demonstrated by their applicability across various platforms, including web and mobile environments. This cross-platform capability ensures that assistive tools can reach a wider audience, catering to users' needs regardless of the device they use.

Real-world examples of successful integration presented in the previous section highlight the impact of this API ecosystem. Feedback from indicates significant improvements in their daily lives, enhancing their independence and empowerment.

In conclusion, the development and deployment of an API ecosystem for assistive technologies offers substantial benefits. By enabling modularity, scalability, and collaborative innovation, this approach not only streamlines the development process but also delivers highly effective tools that make a real difference in users' lives.

VI. CONCLUSION AND FUTURE WORK

The API ecosystem described in this paper was developed with a strong emphasis on collaboration and modularity, leveraging the benefits of a microservices architecture to create highly customizable and adaptable assistive technologies. This collaborative approach is crucial, particularly in academic and publicly funded research, where resources must be used efficiently to avoid redundant implementation and ensure the maximum impact of each project.

The modularity and flexibility of the API ecosystem have enabled rapid development and deployment of various assistive tools, tailored to the unique needs of users with cognitive disabilities. This approach not only reduces development time and costs but also allows for continuous updates and improvements without extensive rework. The successful integration of these APIs into multiple applications demonstrates the potential of microservices for enhancing the scalability and interoperability of assistive technologies.

involve the development of a robust and open API environment. This requires the introduction of a comprehensive API management infrastructure, moving beyond the current reliance on a reverse proxy setup (NGINX server) to a complete API Gateway solution. Such an API Management platform will provide essential features such as versioning, ownership, and SLA policies, which are critical for a mature

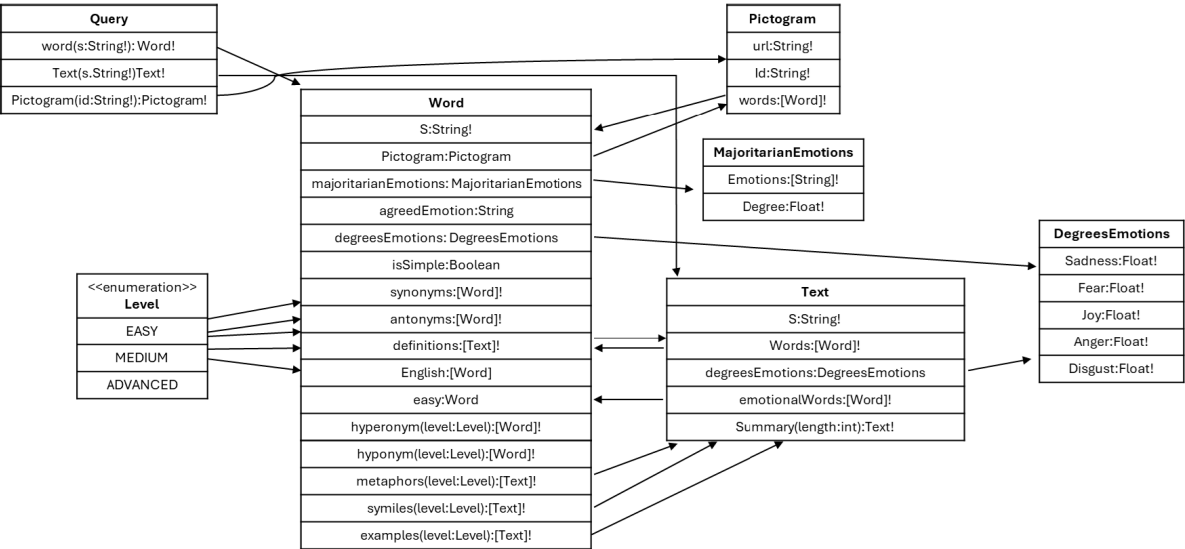


FIGURE 6. Translation of Figure 5. Schema for the GraphQL access layer.

```
GET /nil-ws-api/v1/word/teacher/degree_emotions
→
{
  "Joy": "2.2",
  "Disgust": "1.3",
  "Anger": "1.57",
  "Fear": "1.43",
  "Sadness": "1.3"
}
POST /nil-ws-api/v1/text/emotional_words
{
  "text": "The bees chased me all the way home,
  ↳ but they couldn't sting me because I ran
  ↳ too fast"
}
→
{
  "emotions": {
    "Joy": "2.57",
    "Disgust": "1.69",
    "Anger": "1.61",
    "Fear": "2.35",
    "Sadness": "1.25"
  },
  "words": [
    "bees",
    "chased",
    "home",
    "sting",
    "ran"
  ]
}
```

LISTING 4. Translation of Listing 1. Examples of use of the OpenAPI access layer.

and scalable ecosystem. This component provides scaffolding for building service-based API structure. Usually, an API Manager provides three essential features: a single entry point for all the consumers' requests (API Gateway), a central web-based tool for managing the various policies to be applied, and a marketplace for developers that allows them to easily

```
POST /nil-ws-api/graphql
{
  "query": "{
    word(s: \"bee\") {
      pictogram {
        id
      }
      emotionsDegree {
        Fear
      }
    }
  }"
}
→
{
  "data": {
    "word": {
      "pictogram": {
        "id": "2239"
      },
      "emotionsDegree": {
        "Fear": 3.53
      }
    }
  }
}
```

LISTING 5. Translation of Listing 2. Example of use of the GraphQL access layer. Note how the shape of the request body and response match. The predictability of the response data is a strong point of GraphQL.

find the APIs they need to consume (API Portal). All of these components are normally complemented by a centralized configuration service and an elastic infrastructure for hosting services that implement the API. The potential benefits of developing such an ecosystem are as follows:

- Centralized configuration, a service that all applications use to specify and access their respective configuration information consistently.
- Automatic deployment, a service that invokes and decommissions APIs, and service implementations under administrative control.

Input:

```
{
  "query": "{
    text(text: \"I feel sad when it rains.\") {
      simplified
      emotionsDegree {
        sadness
      }
      pictograms {
        url
      }
    }
  }"
}
```

Response:

```
{
  "data": {
    "text": {
      "simplified": "I feel unhappy when it
        ↪ rains.",
      "emotionsDegree": {
        "sadness": 0.87
      },
      "pictograms": [
        {"url": "pictogram1.png"},
        {"url": "pictogram2.png"}
      ]
    }
  }
}
```

LISTING 6. Translation of Listing 3. Obtaining simplification, sentiment analysis, and pictogram translation of a given text in one request using GraphQL.

- A single security enforcement point is provided by the API Gateway to ensure robust security measures.
- Auditing and monitoring, provided by the API Gateway, enables detailed insights into API usage patterns, the geographical distribution of consumers, and service performance metrics.

The adoption of such infrastructure will not only streamline the management and deployment of APIs but will also provide valuable data to support a continuous improvement cycle for the evolution of APIs. Insights gathered from usage patterns and consumer interactions will help to refine and enhance services, ensuring that they remain relevant and effective.

Furthermore, implementing a comprehensive API management infrastructure will encourage other research groups and developers to contribute new services and expand the ecosystem's functional capabilities. This collaborative expansion is vital for sustained growth and innovation of assistive technologies, ultimately benefiting a wider range of users.

In conclusion, the development of an API ecosystem based on microservices offers significant advantages, particularly in the context of publicly funded research. By avoiding redundant efforts and leveraging shared resources, this approach ensures efficient use of funds and maximizes the impact of research projects. As we move towards a more sophisticated API management framework, we anticipate even greater collaboration and innovation, leading to the creation of more

powerful and adaptable assistive technologies for users with cognitive disabilities.

APPENDIX LISTINGS AND FIGURES IN ENGLISH

Listings 4–6 and Figure 6.

REFERENCES

- [1] I. Gat and G. Succi, "A survey of the API economy," Cutter Consortium, Arlington, MA, USA, Tech. Rep., 2013.
- [2] L. Olson, "The open API economy: What is it and how do I capitalize on it?" in *Proc. Int. Service Technol. Symp.*, 2012, pp. 24–25.
- [3] R. Hervás, V. Francisco, G. Méndez, and S. Bautista, "A user-centred methodology for the development of computer-based assistive technologies for individuals with autism," in *Human-Computer Interaction—INTERACT*. Berlin, Germany: Springer, 2019, pp. 85–106.
- [4] S. Willmott and G. Balas. (2013). *Winning in the API Economy: Using Software and APIs To Transform Your Business, Drive Revenues, Broaden Distribution and Unleash Innovation*. [Online]. Available: <https://www.smallake.kr/wp-content/uploads/2015/02/Winning-in-the-API-Economy-eBook-3scale.pdf>
- [5] F. B. Insights. (2024). *API Management Market Size, Share & Industry Analysis, By Deployment (Cloud and On-premises), By Enterprise Type (Large Enterprises and Small & Medium Enterprises), By Application (Security, Performance Analytics, Governance, Gateway, and Others), By End-user (IT & Telecom, Government, Retail, Healthcare, BFSI, Transport & Logistics, and Others), and Regional Forecast, 2024–2032*. [Online]. Available: <https://www.fortunebusinessinsights.com/api-management-market-108490>
- [6] Trebble. (2023). *The Anatomy of an API*. [Online]. Available: <https://report.treble.com/>
- [7] Uber. *Uber API Developers Documentation Page*. Accessed: Nov. 10, 2019. [Online]. Available: <https://developer.uber.com/docs/riders/references/api>
- [8] Google. *Google Developers Documentation Page*. Accessed: Nov. 10, 2019. [Online]. Available: <https://developers.google.com/>
- [9] M. Bonardi, M. Brioschi, A. Fuggetta, E. S. Verga, and M. Zuccalá, "Fostering collaboration through API economy: The E015 digital ecosystem," in *Proc. IEEE/ACM 3rd Int. Workshop Softw. Eng. Res. Ind. Pract.*, May 2016, pp. 32–38.
- [10] S. Kubler, J. Robert, A. Hefnawy, K. Främling, C. Cherifi, and A. Bouras, "Open IoT ecosystem for sporting event management," *IEEE Access*, vol. 5, pp. 7064–7079, 2017, doi: [10.1109/ACCESS.2017.2692247](https://doi.org/10.1109/ACCESS.2017.2692247). <https://doi.org/10.1109/ACCESS.2017.2692247>
- [11] S.-A. Sansone, P. McQuilton, P. Rocca-Serra, A. Gonzalez-Beltran, M. Izzo, A. L. Lister, and M. Thurston, "FAIRsharing as a community approach to standards, repositories and policies," *Nature Biotechnol.*, vol. 37, no. 4, pp. 358–367, Apr. 2019, doi: [10.1038/s41587-019-0080-8](https://doi.org/10.1038/s41587-019-0080-8).
- [12] *Global Report on Health Equity for Persons With Disabilities*, World Health Org., Geneva, Switzerland, 2022.
- [13] E. Wittern, A. Cha, and J. A. Laredo, "Generating graph QL-wrappers for REST (-like) APIs," in *Proc. Int. Conf. Web Eng.* Cham, Switzerland: Springer, 2018, pp. 65–83.
- [14] R. Bodle, "REGIMES OF SHARING: Open APIs, interoperability, and Facebook," *Inf., Commun. Soc.*, vol. 14, no. 3, pp. 320–337, Apr. 2011.
- [15] *Industry Direction for Open APIs—A Discussion*, IATA, Montreal, QC, Canada, 2017.
- [16] D. Miller, J. Whitlock, M. Gardiner, M. Ralphson, R. Ratovsky, and U. Sarid. (2018). *OpenAPI Specification, Version 3.0.2*. [Online]. Available: <https://spec.openapis.org/oas/v3.0.2>
- [17] Facebook. (2018). *GraphQL Specification*. [Online]. Available: <https://spec.graphql.org/June2018/>
- [18] S. Bautista, R. Hervás, P. Gervás, A. Bagó, and J. García-Ortiz, "Taking text simplification to the user: Integrating automated modules into a web browser," in *Proc. 8th Int. Conf. Softw. Develop. Technol. Enhancing Accessibility Fighting Info-Exclusion*. New York, NY, USA: Association for Computing Machinery, Jun. 2018, pp. 88–96, doi: [10.1145/3218585.3218591](https://doi.org/10.1145/3218585.3218591).
- [19] S. Bautista, R. Hervás, A. Hernández-Gil, C. Martínez-Díaz, S. Pascua, and P. Gervás, "Aratractor: Text to pictogram translation using natural language processing techniques," in *Proc. 18th Int. Conf. Human Comput. Interact.*, Sep. 2017, pp. 1–8.

- [20] H. Overdick, "The resource-oriented architecture," in *Proc. IEEE Congr. Services (Services)*, Jul. 2007, pp. 340–347.
- [21] S. Newman, *Building Microservices: Designing Fine-Grained Systems*. CA, USA: O'Reilly, 2015.
- [22] E. Wolff, *Microservices: Flexible Software Architecture*. Reading, MA, USA: Addison-Wesley, 2016.
- [23] G. A. Miller, "WordNet: A lexical database for English," in *Proc. Human Language Technol., Workshop Held at Plainsboro, NJ, USA*, Mar. 1994, p. 409. [Online]. Available: <https://aclanthology.org/H94-1111>
- [24] A. Martín, R. Hervás, G. Méndez, and S. Bautista, "PICTAR: Una herramienta de elaboración de contenido para personas con TEA basada en la traducción de texto a pictogramas," in *Proc. 19th Int. Conf. Human-Comput. Interact.*, 2018, pp. 53–56.
- [25] E. Agüero and I. Sande. (2019). *Asistente Móvil Para la Interpretación de Texto Dirigido a Personas Con Discapacidad Cognitiva*. [Online]. Available: <https://hdl.handle.net/20.500.14352/15193>
- [26] I. Martín-Berlanga and P. García-Hernández. (2019). *Mejora de la Comprensión Lectora Para la Inclusión Mediante Figuras Retóricas*. [Online]. Available: <https://hdl.handle.net/20.500.14352/15189>
- [27] L. Jiménez-Corta. (2018). *Herramienta De Apoyo a La Navegación Web Para Personas Con Discapacidad*. [Online]. Available: <https://hdl.handle.net/20.500.14352/20592>
- [28] S. González-Álvarez and J. López-Pulido. (2019). *Traductor de Pictogramas a Texto*. [Online]. Available: <https://hdl.handle.net/20.500.14352/15223>
- [29] G. Eugercios, P. Gutiérrez, and E. Kaloyanova. (2018). *Análisis Emocional Para La Inclusión Digital*. [Online]. Available: <https://hdl.handle.net/20.500.14352/20574>
- [30] S. Harper, "Is there design-for-all?" *Universal Access Inf. Soc.*, vol. 6, pp. 111–113, May 2007.



VIRGINIA FRANCISCO received the Ph.D. degree in computer science from the Universidad Complutense de Madrid (UCM), Madrid, Spain, in 2008. She is currently an Associate Professor at the Instituto de Tecnología del Conocimiento (ITC), Department of Software Engineering and Artificial Intelligence, UCM, where she teaches and conducts research in human–computer interaction through natural language user interfaces. She also carries out various initiatives related to instructional innovation.



EUGENIO CONCEPCIÓN received the M.Sc. degree in multimedia from the Universitat Oberta de Catalunya (UOC), Barcelona, Spain, in 2014, and the Ph.D. degree in Computer Science from the Universidad Complutense de Madrid (UCM), Madrid, Spain in 2023. He is currently employed as a Business Innovation Manager at Softtek, an International Technology Company, and also works as an Associated Lecturer at the Facultad de Informática, Department of Software Engineering and Artificial Intelligence, UCM. He has pursued a research-oriented career, collaborating with the NIL research group of the UCM.



ANTONIO F. G. SEVILLA received the degree in computer science and mathematics from the Universidad Autónoma de Madrid (UAM) in 2013, the European master's double degree in language and communication technologies, in 2016, and the Ph.D. degree in computer engineering from the Universidad Complutense de Madrid, in 2023. He is an Assistant Professor at the Facultad de Informática, Department of Software Engineering and Artificial Intelligence, UCM. His research focuses on natural language processing and computational linguistics, with a special interest in sign languages.



GONZALO MÉNDEZ received the Ph.D. degree in computer science from the Universidad Politécnica de Madrid (UPM), Madrid, Spain, in 2008. He is an Associate Professor at the Facultad de Informática, Department of Software Engineering and Artificial Intelligence, UCM, where he is currently the Director. His research interests include computational creativity, narrative generation, natural language generation, intelligent agents, and software architecture.

...



RAQUEL HERVÁS received the Ph.D. degree in computer science from the Universidad Complutense de Madrid (UCM), Madrid, Spain, in 2009. She is currently an Associate Professor at the Facultad de Informática, Department of Software Engineering and Artificial Intelligence, UCM, where she is also the Vice-Dean for studies and quality assurance. Her research interests include human–computer interaction through natural-language user interfaces with a special emphasis on accessibility.